Crack 4-8 LPA Jobs Now !

By Sagar (sagar_mee_)

1) **Focus on building problem-solving speed and accuracy in quantitative aptitude, which often plays a major role in initial rounds**

Quantitative Aptitude Topics:

1. Number Systems
2. Percentages
3. Simple and Compound Interest
4. Ratio and Proportion
5. Average
6. Algebra
7. Time, Speed, and Distance
8. Time and Work
9. Mixtures and Alligations
10. Mensuration
11. Data Interpretation
12. Probability
13. Permutations & Combinations
14. Ages
15. Profit and Loss
16. Simplification and Approximation
17. Series Completion
18. Analogies

1) **Coding Round Preparation**

Most Important Coding Topics

1. **Arrays & Strings** (60-70% of coding questions come from here)

- **Sorting** (Bubble, Selection, Insertion, Quick, Merge)
- **Searching** (Binary Search, Linear Search)
- **Two Pointer Technique**

- **Sliding Window (Kadane's Algorithm for Maximum Subarray)**
- **Basic String Manipulations (Palindrome, Anagrams, Reverse String, Substrings, etc.)**

2. Recursion & Backtracking (Basics)

- Factorial, Fibonacci, Power of a number
- Tower of Hanoi
- N-Queens, Rat in a Maze

3. Linked List (Easy-Medium Level)

- Reverse a Linked List
- Find Middle, Detect Loop
- Merge Two Sorted Lists

4. Stack & Queue (Basic Implementation)

- Stack using Array & Linked List
- Queue using Stack
- Next Greater Element
- Balanced Parentheses

5. Trees & BST (Simple Operations)

- Inorder, Preorder, Postorder Traversal
- Find Height of a Tree
- Lowest Common Ancestor (LCA)
- Level Order Traversal

These are the most important and most repeated previous year questions that company asks :

1. Sum of smallest and largest prime numbers in a range

2.Checking valid email addresses (@gmail.com)

3.Reversing words in a string, checking palindromes

4.Finding missing numbers in a range, combining sorted arrays

5.Validating brackets with a stack (`()[]{}`)

6.Checking if a number is a power of two

7.Calculating trailing zeroes in factorials

8.Sorting algorithms (Bubble sort, Merge sort) for merging array

9.Longest consecutive sequence, first unique character in a string

10.Two-sum problem, moving zeros in arrays

11. Finding duplicate numbers, largest sum in a subarray

2) Technical Knowledge (Java, Python, DBMS)

1.Default thread in JVM, difference between HashMap and Hashtable

2.Heap vs. Stack memory, why Java doesn't use pointers

3.Shallow vs. Deep Copy, Static method/class/variables

4.Constructors vs. Methods, overloading in Java

5.Private, protected, and global attributes in Python

6.Multiple inheritance in Python vs. Java, interpreted vs. dynamically

7.DBMS basics: CRUD operations, role-based access control, primary

8.Discuss project work, focusing on technology stack and problem-approach

3) Computer Networks

- **Networking Basics**: Understand IP addressing, MAC addresses, and basic networking terms.
- **OSI and TCP/IP Models**: Layers of each model and basic functions of each layer.
- **HTTP/HTTPS and Web Protocols**: How web communication works, differences between HTTP and HTTPS.
- **DNS and Load Balancing**: Basics of how domain names are resolved and introductory load balancing concepts.

4) Operating Systems

- **Processes and Threads**: Differences, multithreading, and basic thread concepts.
- **Memory Management**: Stack vs. Heap memory, paging, segmentation.
- **Concurrency**: Synchronization basics, race conditions, deadlocks, and mutexes.
- **File Systems**: Basics of file handling, file system structure, and permissions.

5) Databases

- **SQL Basics**: CRUD operations, joins, aggregations, indexing.
- **Normalization and Keys**: Basic normalization up to 3NF, primary and foreign keys.
- **Transactions**: Understanding ACID properties, how transactions work.
- **NoSQL Basics**: Key-value stores, document databases, differences from relational databases.

6) Object-Oriented Programming (OOP)

- **Basic Concepts**: Encapsulation, inheritance, polymorphism, abstraction.
- **Design Patterns**: Basic patterns like Singleton, Factory, Observer.
- **SOLID Principles**: High-level understanding of SOLID (Single Responsibility, Open-Closed, etc.).
- **Practical Applications**: Apply OOP concepts in projects, understand how OOP is used in real-world software.

ALL THE BEST 💔